

Project Structure

Version: 1.0

Purpose

This document defines the physical repository and source-code structure of Trading Platform Pro and its primary application, the Trading Cockpit.

The project structure shall reflect:

- architecture boundaries
- domain capability boundaries
- vertical product development
- explicit technical responsibilities
- maintainable repository growth

The repository shall remain easy to navigate.

Avoid directory structures created only for hypothetical future requirements.

Repository Layout

```
```text
project/
■
■■■ docs/
■■■ src/
■■■ tests/
■■■ config/
■■■ scripts/
■■■ tools/
■■■ temp/
■■■ .github/
■■■ pyproject.toml
■■■ requirements.txt
■■■ requirements-dev.txt
■■■ requirements-docs.txt
■■■ README.md
■■■ AGENTS.md
```
```

Each top-level directory shall have an explicit responsibility.

Source Code

All production code resides in:

```
```text
src/
```
```

Primary Python package:

```
```text
src/
■■■ trading_platform/
```
```

The source structure combines architecture layers with explicit capability boundaries.

Primary Source Structure

Target structure:

```
```text
src/
 ■■■ trading_platform/
 ■
 ■■■ domain/
 ■■■ application/
 ■■■ infrastructure/
 ■■■ presentation/
 ■■■ shared/
```
```

Architecture layer responsibilities are defined in `Architecture.md`.
The physical structure shall preserve dependency direction.

Domain Structure

Domain code is organized by business capability.

Target structure:

```
```text
domain/
 ■
 ■■■ instruments/
 ■■■ watchlists/
 ■■■ trading_candidates/
 ■■■ trading_decisions/
 ■■■ portfolio/
 ■■■ positions/
 ■■■ orders/
 ■■■ risk/
 ■■■ trading_review/
```
```

A capability package may contain:

```
```text
orders/
 ■
 ■■■ entities.py
 ■■■ value_objects.py
 ■■■ events.py
 ■■■ services.py
 ■■■ exceptions.py
 ■■■ repositories.py
```
```

Files shall only be introduced when required.

Do not create empty architectural placeholder files without implementation need.

Domain packages shall not depend on infrastructure or presentation.

Application Structure

Application code is organized around use cases and workflow coordination.

Target structure:

```
```text
application/
 ■
 ■■■ instruments/
 ■■■ watchlists/
 ■■■ trading_candidates/
 ■■■ trading_decisions/
```
```

```
■■■ portfolio/
■■■ positions/
■■■ orders/
■■■ risk/
■■■ trading_review/
---
```

A capability package may contain:

```
```text
orders/
■
■■■ commands.py
■■■ queries.py
■■■ services.py
■■■ ports.py
■■■ dto.py

```

Application packages coordinate use cases.  
Application packages shall not contain presentation code.

---

## Infrastructure Structure

Infrastructure is organized by technical capability.

Target structure:

```
```text
infrastructure/
■
■■■ bootstrap/
■■■ config/
■■■ logging/
■■■ persistence/
■■■ broker/
■■■ market_data/
■■■ messaging/
■■■ runtime/
■■■ scheduler/
■■■ monitoring/
■■■ health/
■■■ files/
■■■ serialization/
■■■ security/
---
```

Infrastructure packages implement technical capabilities and external adapters.
Infrastructure shall not contain trading decisions or domain business rules.

Broker Infrastructure

Broker-specific implementations reside under:

```
```text
infrastructure/
■■■ broker/

```

Example:

```
```text
broker/
■
■■■ base/
```

```
■■■ interactive_brokers/
---
```

A provider-specific package may contain:

```
```text
interactive_brokers/
■
■■■ adapter.py
■■■ connection.py
■■■ order_mapper.py
■■■ position_mapper.py
■■■ models.py
■■■ errors.py
```
```

Provider-specific structures shall only be created when implementation requires them.
Broker APIs shall not leak into Domain packages.

Market Data Infrastructure

Market data implementations reside under:

```
```text
infrastructure/
■■■ market_data/
```
```

Example:

```
```text
market_data/
■
■■■ base/
■■■ providers/
```
```

Provider packages may contain:

```
```text
provider_name/
■
■■■ adapter.py
■■■ connection.py
■■■ subscription.py
■■■ mapper.py
■■■ errors.py
```
```

Market data provider types shall not become Domain models.

Persistence Infrastructure

Persistence implementations reside under:

```
```text
infrastructure/
■■■ persistence/
```
```

Target structure may include:

```
```text
persistence/
■
■■■ database/
■■■ models/
■■■ repositories/
```

```
■■■ mappings/
■■■ migrations/

```

SQLAlchemy-specific code shall remain inside Infrastructure.  
Persistence models shall remain distinguishable from Domain models.

---

## Presentation Structure

Trading Cockpit presentation code resides under:

```
```text
presentation/
---
```

Target structure:

```
```text
presentation/
■
■■■ app/
■■■ workspace/
■■■ widgets/
■■■ context/
■■■ commands/
■■■ notifications/
■■■ themes/
■■■ common/

```

Presentation packages shall not access concrete Infrastructure adapters directly.  
Presentation shall not contain domain business rules.

---

## Application Shell

Application shell components reside under:

```
```text
presentation/
■■■ app/
---
```

Responsibilities may include:

- main window
- application startup presentation
- application shutdown presentation
- status bar
- top-level navigation

Example:

```
```text
app/
■
■■■ main_window.py
■■■ navigation.py
■■■ status_bar.py

```

---

## Workspace Structure

Workspace presentation components reside under:

```
```text
presentation/
■■■ workspace/
```

```
'''
```

Responsibilities may include:

- workspace manager
- docking coordination
- widget registry
- workspace restoration
- workspace persistence coordination

Example:

```
'''text
```

workspace/

```
■
```

```
■■■ workspace_manager.py
```

```
■■■ widget_registry.py
```

```
■■■ workspace_state.py
```

```
■■■ docking.py
```

```
'''
```

Workspace state shall remain separate from business persistence.

```
---
```

Widget Structure

Widgets reside under:

```
'''text
```

presentation/

```
■■■ widgets/
```

```
'''
```

Widgets should be organized by product capability.

Target structure:

```
'''text
```

widgets/

```
■
```

```
■■■ market/
```

```
■■■ charts/
```

```
■■■ decision_center/
```

```
■■■ portfolio/
```

```
■■■ risk/
```

```
■■■ trading/
```

```
■■■ reporting/
```

```
■■■ workspace/
```

```
'''
```

Example:

```
'''text
```

widgets/

```
■■■ trading/
```

```
■■■ order_entry/
```

```
■■■ order_review/
```

```
■■■ open_orders/
```

```
■■■ execution_monitor/
```

```
'''
```

A widget package may contain:

```
'''text
```

order_entry/

```
■
```

```
■■■ widget.py
```

```
■■■ view_model.py
```

```
■■■ models.py
```

```
■■■ delegates.py
```

Only required files shall be introduced.
Widgets shall follow `Widget\_Catalog.md`.

Shared Instrument Context Structure

Shared Instrument Context presentation code resides under:

```
```text
presentation/
■■■ context/
```
```

Example:

```
```text
context/
■
■■■ instrument_context.py
■■■ context_events.py
■■■ context_binding.py
```
```

Shared context shall not become unrestricted global mutable state.
Context ownership and propagation shall remain explicit.

Shared Code

The `shared/` package is reserved for code with demonstrated cross-capability reuse.

Example:

```
```text
shared/
■
■■■ types/
■■■ utilities/
```
```

The `shared/` package shall remain small.
Do not move code into `shared/` solely because its ownership is unclear.
Before introducing shared code verify:

- at least two real consumers exist
- responsibility is technical or genuinely cross-capability
- no domain boundary is weakened

`shared/` shall not become a miscellaneous package.

Vertical Product Slices

A product increment may modify multiple architecture layers.

Example:

```
```text
Watchlist Workflow
■
■■■ domain/watchlists/
■■■ application/watchlists/
■■■ infrastructure/persistence/repositories/
■■■ presentation/widgets/market/watchlist/
■■■ tests/
■■■ docs/
```
```

This is expected.
Vertical product development does not mean mixing layer responsibilities.

Each implementation remains in the appropriate architecture layer.

Documentation Structure

All authoritative documentation resides under:

```
```text
docs/
```
```

Target structure:

```
```text
docs/
■
■■■ product/
■■■ architecture/
■■■ specifications/
■■■ api/
■■■ ui/
■■■ operations/
■■■ developer/
■■■ user/
■■■ decisions/
■■■ generated/
```
```

Each documentation directory has one primary responsibility.
Markdown files are the documentation source of truth.

Generated Documentation

Generated documentation resides under:

```
```text
docs/generated/
```
```

Structure:

```
```text
generated/
■
■■■ docx/
■■■ pdf/
```
```

Generated files are derived from Markdown source documents.

Generated files shall not be edited manually.

The documentation generation pipeline is implemented in:

```
```text
scripts/generate_docs.py
```
```

Product Documentation

Product documents reside under:

```
```text
docs/product/
```
```

Examples:

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md

Product documentation defines product direction and workflow priorities.

Architecture Documentation

Architecture documents reside under:

```
```text
docs/architecture/
```
```

Examples:

- Architecture.md
- Domain_Model.md
- Infrastructure.md
- Project_Structure.md

Architecture documentation defines structural and dependency boundaries.

Specification Documentation

Technical and product specifications reside under:

```
```text
docs/specifications/
```
```

Examples:

- Technical_Specifications.md
- Widget_Catalog.md

Specifications define implementation and product capability requirements.

API Documentation

API documentation resides under:

```
```text
docs/api/
```
```

Examples:

- API_Guidelines.md
- API_Specification.md
- Data_Models.md
- Error_Codes.md

UI Documentation

UI documentation resides under:

```
```text
docs/ui/
```
```

Examples:

- UI_Guidelines.md
- Accessibility.md
- Keyboard_Shortcuts.md

Operations Documentation

Operational documentation resides under:

```
```text
docs/operations/
```
```

Examples:

- Deployment.md
- Monitoring.md
- Operations_Manual.md
- Security.md

Developer Documentation

Developer documentation resides under:

```
```text
docs/developer/
```
```

Examples:

- Coding_Standards.md
- Development_Guidelines.md
- Testing_Strategy.md
- Configuration.md
- Runtime.md
- Logging.md

User Documentation

User documentation resides under:

```
```text
docs/user/
```
```

Examples:

- User_Manual.md
- Quick_Start.md
- Troubleshooting.md

Decision Documentation

Decision and roadmap documentation resides under:

```
```text
docs/decisions/
```
```

Examples:

- Decision_Log.md
- Roadmap.md
- Change_Log.md

Significant architectural decisions may additionally use ADR files where required.

Tests

Automated tests reside under:

```
```text
tests/
```
```

Primary test structure:

```
```text
tests/
■
■■■ unit/
■■■ integration/
```
```

```
■■■ system/
■■■ performance/
---
```

Tests should mirror production capability boundaries where practical.

Example:

```
```text
tests/
■■■ unit/
■■■ domain/
■ ■■■ orders/
■■■ application/
■■■ orders/

```

Critical trading workflows require regression protection.

---

## Configuration

Configuration files reside under:

```
```text
config/
---
```

Configuration shall remain externalized and environment-aware.

Possible environments:

```
```text
config/
■
■■■ development/
■■■ testing/
■■■ paper/
■■■ live/

```

PAPER and LIVE configuration shall remain explicitly distinguishable.

Secrets shall not be stored in source-controlled configuration.

---

## Scripts

Reusable project automation resides under:

```
```text
scripts/
---
```

Examples:

- documentation generation
- build automation
- release automation
- maintenance utilities

Scripts shall not contain application business logic.

Tools

Developer and repository tools may reside under:

```
```text
tools/

```

Tools are not production application components.

Tool dependencies shall not leak into production runtime unless explicitly required.

---

# Temporary Files

Temporary local working files may reside under:

```
```text
temp/
```
```

`temp/` is intended for development and temporary processing artifacts.  
Production application behaviour shall not depend on files under `temp/`.

---

# GitHub Automation

Repository automation resides under:

```
```text
.github/
```
```

Structure may include:

```
```text
.github/
■■■ workflows/
■■■ ISSUE_TEMPLATE/
■■■ PULL_REQUEST_TEMPLATE/
```
```

GitHub Actions may provide:

- quality checks
- automated tests
- documentation generation
- release automation

---

# Naming Conventions

Directories:

- lowercase
- snake\_case where multiple words are required
- descriptive names
- singular or plural according to capability meaning

Python files:

- snake\_case

Classes:

- PascalCase

Functions and methods:

- snake\_case

Constants:

- UPPER\_CASE

Tests:

```
```text
test_.py
```
```

---

# File Responsibility

Each file shall have one clear primary responsibility.

Avoid large generic files such as:

- helpers.py
- utils.py
- common.py

- manager.py

unless the responsibility is explicitly defined and documented.  
Prefer domain or technical language that identifies actual ownership.

---

## Package Responsibility

Each package shall represent:

- a business capability
- an architecture responsibility
- a technical infrastructure capability
- a defined presentation responsibility

Avoid packages created only to group unrelated files.

---

## Structure Evolution Rules

When introducing a new directory or package:

1. Identify the product or technical requirement.
2. Identify the owning architecture layer.
3. Identify the capability or technical responsibility.
4. Check whether an existing package already owns the responsibility.
5. Avoid duplicate structures.
6. Update documentation where structural boundaries change.

Do not create directory trees for future features before implementation requires them.

---

## Repository Growth Principles

As the project evolves:

- extend existing capability boundaries where appropriate
  - preserve dependency direction
  - avoid parallel implementations
  - avoid speculative shared packages
  - keep directory depth practical
  - keep responsibilities explicit
  - review repository structure regularly
- Repository structure shall evolve with implemented product workflows.

---

## Structure Review Checklist

Before introducing or changing project structure verify:

- product or technical requirement identified
- architecture layer identified
- capability owner identified
- existing location cannot be reused
- dependency direction preserved
- no duplicate responsibility introduced
- no speculative package introduced
- `shared/` usage justified
- test location defined
- documentation location defined
- documentation updated

---

## Related Documents

- Product\_Vision.md
- Product\_Roadmap.md

- Project\_Overview.md
- Architecture.md
- Domain\_Model.md
- Infrastructure.md
- Technical\_Specifications.md
- Widget\_Catalog.md
- Testing\_Strategy.md
- Coding\_Standards.md
- AGENTS.mdClean